Flow Matching for Molecular Generation

Thesis Report 1

Nico Conti
Università di Pisa

May 30, 2026

1 Code bases referenced

The following code repositories were consulted as references for how to implement the various components:

- `github.com/Asduffo/FreeGress` (built on `github.com/cvignac/DiGress`): referenced to understand how guidance is introduced, and used to build the graph-transformer backbone on top of it. The QM9 and ZINC-250k preprocessing was also referenced.
- `github.com/facebookresearch/flow_matching` referenced for the flow-matching objective, path construction, and ODE sampling.
- CatFlow / VFM reference code (paper supplementary material): referenced for the graph-transformer layers and masking utilities. The authors also build a vanilla flow-matching loss on top of this backbone, which they run as a baseline. The QM9 and ZINC-250k preprocessing was also referenced.

My code: `github.com/Nico-Conti/flow-matching-molecules`.

2 Data provenance and cleaning

2.1 Source

Both datasets follow the same path. The raw molecules, QM9 and ZINC-250k, are obtained through `torch_molecule` (`load_qm9`, `load_zinc250k`), passed through the custom cleaning pipeline of Section 2.2, and saved as personal Hugging Face datasets: `nico8771/qm9_clean` and `nico8771/zinc_clean`. QM9 keeps the dataset's DFT properties (μ, α, HOMO, LUMO, gap, c_v), whereas the ZINC targets ($\log P$, QED, SAS) are recomputed with RDKit from the cleaned SMILES.

2.2 The cleaning pipeline

1. **Parse**: transform raw SMILES into molecules via RDKit and drop unparseable strings.
2. **Standardize**: normalize the molecule by stripping stereochemistry, optionally neutralizing charges, and running RDKit's `SanitizeMol`.
3. **Canonicalize:** re-emit the molecule as a SMILES string s_{clean} with RDKit's canonical ordering, so the strings are in a fixed order for the round-trip consistency check.
4. **Featurize**: re-parse s_{clean}, kekulize it (converting aromatic rings to explicit single/double bonds), and build the one-hot node and edge tensors. Atoms outside the vocabulary defined below, or molecules that cannot be kekulized, are dropped.

5. **Round-trip filter**: decode the tensors back to a molecule and keep it only if it reproduces s_{clean} exactly. This guarantees the graph representation faithfully encodes the cleaned molecule.

The atom vocabulary is charge-aware and dataset-specific:

$$\text{QM9: } \{\text{C}, \text{N}, \text{O}, \text{F}\}, \qquad \text{ZINC: } \{\text{C}, \text{N}, \text{O}, \text{F}, \text{P}, \text{S}, \text{Cl}, \text{Br}, \text{I}, \text{N}^+, \text{O}^-\}.$$

For ZINC, the non-neutral groups N^+ and O^- are kept as standalone atom types rather than neutralized, matching the FreeGress convention. Bonds use four classes: none / single / double / triple.

Dataset	Input	Kept	Dropped	Retention
QM9	133,885	132,040	1,845	98.6%
ZINC	249,455	247,449	2,006	99.2%

2.3 Discrepancy with the FreeGress preprocessing

FreeGress keeps about 228k of the roughly 249k ZINC molecules, an 8% drop of around 21k records (the paper reports it per split). Looking at their repo (still not sure if this is an official repo), they store aromatic rings with a dedicated `AROMATIC` bond class rather than kekulizing them via RDKit. On reconstruction `SanitizeMol` must kekulize those bonds, and it fails on ambiguous cases, raising a `KekulizeException` that drops the molecule.

I confirmed this by replicating their representation in my own pipeline: storing bonds with the aromatic class and rebuilding drops 17,712 of 247,449 cleaned ZINC molecules (7.16%), all to `KekulizeException`, which matches their reported loss. I instead opt for the approach seen in the CatFlow repo, which kekulizes up front at featurization, storing explicit single/double bonds and no aromatic class. Reconstruction then only re-perceives aromaticity from those bonds, so it never has to kekulize, and the drop disappears: the final pipeline keeps about 99%.

3 Model architecture

The initial model, built for a preliminary study in a non-CFG setting and following DiGress, CatFlow, and Flow Matching code, is a graph transformer trained with a vanilla continuous flow-matching objective.

Three coupled streams. It carries and refreshes, at every layer:

- Node features $\mathbf{X} \in \mathbb{R}^{n \times k_X}$;
- A dense edge tensor $\mathsf{E} \in \mathbb{R}^{n \times n \times k_E}$;
- A global vector y.

Layer stack. It stacks **5** transformer layers (`XEyTransformerLayer`); each layer:

- Runs a `NodeEdgeBlock` self-attention;
- Wraps it in a residual + LayerNorm + feed-forward on all three streams;
- Pools nodes and edges back into y (mean/min/max/std).

FiLM conditioning. Feature-wise linear modulation in three directions:

- $\mathsf{E} \to \mathbf{X}$: Edges scale and shift the query–key attention scores, making attention bond-aware.
- $y \to \mathsf{E}$: The global vector modulates the edge update.
- $y \to \mathbf{X}$: The global vector modulates the node update.

Conditioning vector y. For now the only conditioning signal is a sinusoidal embedding of the flow time $t \in [0, 1]$.

I observed that several extra features could be exploited here, but whether they are usable depends, for most of them, on whether the intermediate state at each time t can be read as a well-defined graph/adjacency. I leave these out for initial testing.

Flow-matching objective. The objective comes from regressing the velocity of a conditional-OT probability path:

- Training minimizes a masked mean-squared error between the predicted and target velocity.
- The target is the conditional vector field of a linear conditional-OT path: for each example a Gaussian noise point $\mathbf{X}_0$ is drawn and the target velocity is the displacement $\mathbf{X}_1 - \mathbf{X}_0$, which stays constant along the path.
- The prediction $(\hat{v}_{\mathbf{X}}, \hat{v}_{\mathsf{E}})$ is produced by the graph transformer from the noised graph $(\mathbf{X}_t, \mathsf{E}_t)$ and the time t, and is regressed toward this target velocity.

4 Evaluation and sampling

Evaluation generates N molecules, decodes each to a canonical SMILES, and scores them with three metrics; a molecule is valid if it yields a canonical SMILES:

$$\text{validity} = \frac{\#\,\text{valid}}{N}, \qquad \text{uniqueness} = \frac{\#\,\text{unique valid}}{\#\,\text{valid}}, \qquad \text{novelty} = \frac{\#\,\text{unique valid not in train}}{\#\,\text{unique valid}}. \tag{1}$$

Novelty is measured against the training SMILES set.

To sample, the atom count of each molecule is drawn from a histogram of the dataset's atom-count distribution, and the learned field is integrated from Gaussian noise to $t = 1$ with 100 Euler steps.

5 Initial results

Results are from an unconditional run on QM9.

VUN and FCD. Evaluation generates 10,000 molecules (100 Euler steps).

Model	Validity	Uniqueness	FCD↓
CatFlow	99.8	99.9	0.44
Dirichlet FM	99.1	98.2	0.89
DiGress	99.0	96.2	—
Vanilla flow matching (paper)	94.1	98.2	5.16
Flow matching (ours)	88.4	93.8	2.03

The baseline figures are reproduced from CatFlow's Table 2 (Eijkelboom et al. 2024).

Training. The total loss converges to a rolling mean of ≈ 0.72 ($\sigma/\mu \approx 7\%$), with validation loss ≈ 0.72 under EMA weights.

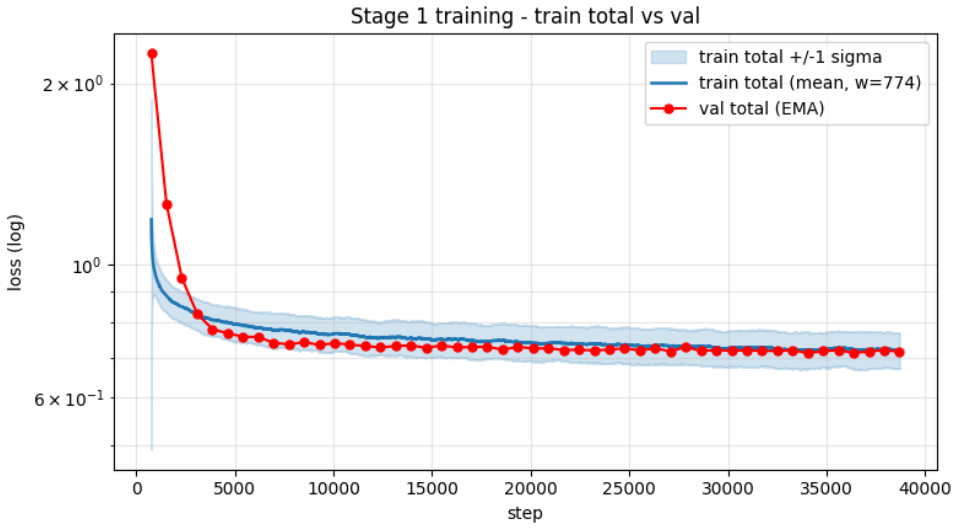

Figure 1: QM9 training: rolling-mean loss with $\pm 1\sigma$ band.

Generated molecules. Using RDKit's `Draw` method, we can visualize some of the generated molecules.

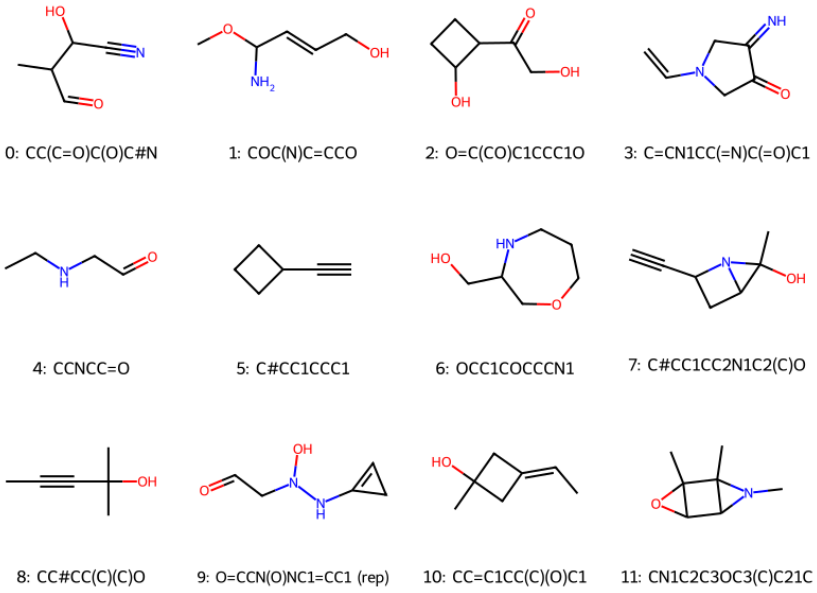

Figure 2: Examples of generated QM9 molecules.

Note on ZINC-250k. Results on ZINC-250k are not reported here due to not having enough compute time on the Colab free plan.